Factory-Line Digital Twin Formula Modules

Research perspective: testing reusable mathematical expression execution in a precisely defined application kernel.

Research question

Can the repeatable mathematical kernel used in factory-line digital twin formula modules be executed, reused and maintained with measurable advantages while preserving domain-specific accuracy?

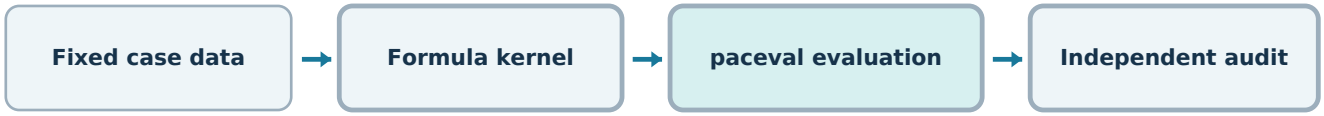
Scientific context and purpose

The proposed work isolates a well-defined mathematical calculation in factory-line digital twin formula modules. The central experimental question is whether executing a repeatedly reused expression with paceval improves a real workflow while preserving the original domain model, units, numerical tolerances and input data. This is a research proposal, not a report of measured performance or customer deployment.

Research at a glance

Target application	Factory-Line Digital Twin Formula Modules
Mathematical task	Reusable expression evaluations; not a replacement for the underlying model or solver.
Pilot workload	Public synthetic line-event trace and twenty interchangeable local model expressions
Primary measures	Step time, output agreement and expression-change burden
Current evidence	Hypothesis; no use-case-specific paceval benchmark is reported.

PROPOSED EVALUATION WORKFLOW



All model assumptions and datasets are held fixed between implementations.

Figure 1. Comparison architecture for an independently reproducible first experiment. This is a proposed setup, not a production deployment.

1. Application-specific mathematical experiment

The proposed pilot addresses factory-line digital twin formula modules. The starting expression below is an intentionally simplified model that enables an exact benchmark boundary. It is not asserted to be a validated physical, financial or clinical model.

Illustrative mathematical kernel

```
throughput=rate*(1-downtime_fraction)
```

The expression is a notation-level research starting point: if the literal syntax is unsupported by a specific paceval build, an algebraically equivalent expression must be agreed and then used unchanged in both comparison implementations. Summation, temporal state and optimization are handled by the same external program unless explicitly benchmarked as subfunctions.

Dataset, assumptions and workload

Public synthetic line-event trace and twenty interchangeable local model expressions

Define units, allowed input intervals, treatment of missing inputs, parameter constraints, initial conditions and a reference answer before examining performance results. Where publicly available data are mentioned, access conditions and licenses must be checked first.

Scope of paceval integration

Generate a reusable computation handle for the mathematical formula, evaluate new parameter sets and batch inputs, and compare against the same expression implemented in an established baseline. Preserve downstream decisions, numerical solvers, simulation steps, random-number paths and objective definitions.

Out-of-scope systems and implicit confounders

Queues, breakdown distributions and discrete-event logic are not expressible as one scalar formula.

A faster mathematical kernel does not establish domain-model validity; the application-level result must be independently checked.

2. Experimental work packages

A. Build the reference workload

Prepare the defined workload: Public synthetic line-event trace and twenty interchangeable local model expressions. Freeze inputs, initial conditions, random seeds, data splits, numerical types and algorithm choices before timing. Record complete setup instructions.

B. Compare current implementations

Reference path: Native formulas in the same discrete-event simulator. Experimental path: identical formula semantics with persistent paceval computation objects, run in scalar and batch mode.

Create formula computation objects once where valid, reuse them for new input values, and measure scalar and batch modes. Retain the incumbent orchestration, solvers, neural inference, sampling, search and data-processing steps unchanged.

C. Verify correctness independently

Use an independent high-precision or analytically derived oracle for edge cases, and report absolute/relative error, nonfinite values, domain violations and cases near cancellation or thresholds.

D. Account for the whole application

For each case record setup, graph creation, input conversion, formula execution, output transfer, all remaining application steps and total elapsed time. Repeat after warm-up; vary formula count, calls per formula and batch size; record thread count, CPU model, energy method and compiler settings.

Preparation cost	Create and validate formula handles once; capture compilation or graph-building overhead.
Repeated evaluation	Measure scalar and vectorized execution against a numerically matched native reference.
End-to-end application	Step time, output agreement and expression-change burden
Numerical audit	Check an independent oracle and record outliers, invalid values and failure modes.
Change management	Modify a declared formula and measure actual engineering steps and regression tests.

An important outcome is a *negative* result if the incumbent implementation is already faster, easier to maintain or more robust. That is scientifically useful evidence about the boundary of the proposed method.

3. Evaluation, limitations and decisions

Pre-registered measurements

Step time, output agreement and expression-change burden

Report throughput (accepted evaluations per second), median and 99th-percentile latency, peak resident memory and full workflow time. For a relevant energy comparison, meter joules per accepted result on the same hardware under documented power conditions. Report preparation and update effort separately from runtime.

Numerical quality requirements

Use an independent high-precision or analytically derived oracle for edge cases, and report absolute/relative error, nonfinite values, domain violations and cases near cancellation or thresholds.

Check normal inputs as well as near-zero denominators, threshold boundaries, nonfinite values and extreme parameter scales. Tests on a finite grid are not proofs of global error bounds. Optional interval-mode output must be checked against an independent rigorous enclosure before any claim of certified intervals.

Go / no-go criterion

Proceed if numerical results and domain metrics remain within pre-registered tolerances and either the measured full workflow is at least 10% faster or the audited expression-change effort decreases at least 30%, without exceeding resource/latency limits. Otherwise retain the incumbent implementation.

Break-even and sensitivity

If the native reference costs t_native per call, the experimental implementation costs t_graph per call and one-time graph construction costs T_create, the idealized reuse break-even is N > T_create/(t_native - t_graph), only when t_graph < t_native. Real measurements must additionally include conversions, scheduling and data transport. Do not interpret this formula as a measured speedup.

Scientific and commercialization limitations

Queues, breakdown distributions and discrete-event logic are not expressible as one scalar formula. A faster mathematical kernel does not establish domain-model validity; the application-level result must be independently checked.

The anticipated advantage may be faster repeated evaluation, lower formula-maintenance effort, or a clearer numerical audit. Each benefit is only a hypothesis until independently measured. No percentage speedup, energy reduction or customer-dollar value is asserted by this proposal.

Path to an experimental paper

The present document is the first research proposal. A later experimental paper would need actual execution logs, independent source code review, controlled baseline workloads and uncertainty ranges. Numerical performance alone is not proof of scientific validity or customer adoption.

4. Reproducibility plan and selected sources

To convert this proposal into a research result, the study will publish a permitted dataset manifest, deterministic seeds, version-pinned model expressions, data pre-processing scripts, hardware/compiler configurations, raw timing records, calculation outputs and a cross-reference between each table and the generating script.

Minimal publication checklist

- Pre-register formula and units, data subset, test split and fixed external algorithms.
- Document scalar and batch APIs, computation object lifetime and cache invalidation.
- Use identical numerical precision, math function semantics and algorithmic stopping criteria.
- Execute and record cold, warm, and formula-update benchmarks with multiple repeated trials.
- Compare independent scientific outputs, not merely the mathematical kernel values.
- Describe excluded cases, invalid inputs and all failed runs instead of selectively reporting gains.
- Publish negative results and the precise observed break-even range.

Selected technical resources

1. paceval mathematical engine: public C/C++ API, reusable computations and batches

https://github.com/paceval/paceval
2. Reproducible evaluation design: Hansen et al., COCO benchmarking methodology (2021)

https://doi.org/10.1080/10556788.2020.1808977
3. Functional Mock-up Interface specification

https://fmi-standard.org/docs/main/
4. Modelica modeling and simulation language specification

https://specification.modelica.org/

Research collaboration

The proposed study is suitable as a small open numerical pilot. It can subsequently be extended to partner-provided workloads subject to the relevant data, security, publication and licensing terms. No customer endorsement or deployment is implied.

Status: independent research proposal. Figures, statements of prospective performance and application benefits are hypotheses unless and until the specified experiments are completed.